Before dropping Unused Indexes, spend some time to double check that they aren't used.

- Run the unused indexes query, identifying indexes with no Index Scans
- As a rollback plan precaution, copy the `CREATE INDEX` definition prior to removal so that it's available to recreate if needed.
- Perform the index removal using the `CONCURRENTLY` option

If you're removing an Index via an Active Record Migration, perform the index removal conditionally in case the index doesn't exist for all databases where the migration runs. This can be the case if you have many deployments of your application each with their own database, and the database definition has drifted.

To collect the Index definition as a CREATE INDEX statement, run the following query from psql with an example Rideshare table index.

sql/index_create_statements.sql

```
SELECT indexdef FROM pg_indexes
WHERE indexname = 'index_users_on_email';
```

An Active Record Migration that conditionally removes the index might look like this:

ruby/remove_index_concurrently.rb

```
class RemoveIndexUsersEmail < ActiveRecord::Migration[7.1]
  disable_ddl_transaction!

  def change
    if index_exists?(:users, :email)
      remove_index(:users, :email, algorithm: :concurrently)
    end
  end
end
```

With Active Record, if the index cannot be found using the table and field name, the `name` option can be used to explicitly name the index.

Besides Unused Indexes, another form of Index Maintenance is to prune Duplicate or Overlapping Indexes. What do those look like?

Removing Duplicate And Overlapping Indexes

Indexes cover one or more columns, and PostgreSQL does not prevent creating multiple indexes covering the same columns as long as the indexes have unique names. Try this out in psql by running these statements:

```
CREATE INDEX index_users_on_email2 ON users (email);
CREATE INDEX index_users_on_email3 ON users (email);
```